

APPENDIX

A	Task Evaluation Subgoals	A2
B	Prompt Details	A2
C	Details of Demonstration Collection Failures	A7
D	Policy Execution Failure Analysis	A8
E	Details of the TAMP and Teleoperation Portions	A8

A. Task Evaluation Subgoals

We score evaluation rollouts against the binary subgoals in Table II. Each task has an ordered list of independent subgoals, and after watching a rollout a human checks off the ones it met; a rollout is a success when it meets every subgoal for its task, and its task progress is the fraction of subgoals met when the rollout terminates. We report the mean progress and the success rate over rollouts. Every approach is scored on the same rubric for a given task, and the language instruction quoted under each task heading is the prompt to the policy at the start of each rollout.

Subgoal	Criterion
Cover Bread Rolls	
<i>"place the 3 breads on the plate and then cover the plate with the cloth"</i>	
one_bread	One piece of bread ends up on the plate
two_bread	A second piece of bread ends up on the plate
three_bread	All three pieces of bread end up on the plate
cover_plate	The plate is covered with the cloth
Solve Constrained Puzzle	
<i>"place the toy on the cloth and solve the puzzle"</i>	
place_toy_on_cloth	The toy is picked up and placed on the cloth
solve_the_puzzle	The puzzle is solved, with each piece seated in its matching cut-out
Sort & Cover Snacks	
<i>"first, place the bread in the blue bowl. second, place the fruit in the green bowl and cover the blue bowl with the cloth"</i>	
place_bread	The bread ends up in the blue bowl
place_fruit	The fruit ends up in the green bowl
cover_blue_bowl	The blue bowl is covered with the cloth
Open Obstructed Book	
<i>"take the marker off the book and place it in the wooden tray, then open the book"</i>	
take_the_marker	The marker is taken off the book
place_marker_in_tray	The marker ends up in the wooden tray
open_book	The book is opened
Store Bread in Closed Box	
<i>"place the bread on the plate, and then open the box and place the bread in the box"</i>	
place_bread_plate	The bread is placed on the plate
open_box	The box is opened
place_bread_box	The bread ends up in the box

TABLE II: Task-specific evaluation subgoals and success criteria, with the language instruction given to the policy for each task.

B. Prompt Details

a) Detection and goal translation: It returns the object names and everything downstream is written over, so it is also the prompt that fixes the vocabulary the plan may use.

Perform two tasks on this image based on the task instruction:

`"{task_instruction}"`.

TASK 1 - OBJECT DETECTION:

Detect and return bounding boxes for objects in the image.

- DO NOT include the robot, robot gripper, or table surface
- DO NOT include objects or surfaces irrelevant to the task or too far away to matter (e.g. walls, things on the wall that are far away, things on the floor below the table, people who might be in the scene, etc.)
- Limit to 25 objects
- If an object appears multiple times, name them by unique characteristics (color, size, position, etc.). If they seem the same, then just use numbers (e.g. 'soda_can1' and 'soda_can2', ... for identical-looking soda cans)
- An object **cannot** have the same name as another under any circumstance.
- Format: normalized coordinates 0-1000 as integers

Be very careful to identify objects and name them in a way that's relevant to the task. If the task involves picking up a red apple, make sure that 'red' appears in the name of the apple.

`{extra_objects}`

TASK 2 - TASK TRANSLATION:

Translate this natural language instruction into some conjunction of formal predicates:

AVAILABLE PREDICATES:

- `on(movable, surface)`: Object A is placed on top of object B
- `holding(movable)`: The robot is holding the named object

It is very important that the goal is exact: use your visual recognition, common-sense and reasoning abilities to make sure the goal expression is perfectly accurate.

For instance - for the task "throw away the trash in the bin" when there is a bin, an open empty chips packet, an empty soda can, a closed and full soda bottle, and several full candy bars on the table, the goal should be:

```
"predicates": [  
  {"name": "on", "args": ["chips_packet", "bin"]},  
  {"name": "on", "args": ["soda_can", "bin"]},  
]
```

This is because only the chips and soda are empty and clearly trash. Everything else is still usable!

Another example - for the task "pick up the apple" when there is a red apple, a banana, and a mug on the table, the goal should be:

```
"predicates": [  
  {"name": "holding", "args": ["red_apple"]},  
]
```

This is because the task is to pick up/grab/hold an object, not to place it somewhere.

Return a single JSON object with this structure (no code fencing):

```
{  
  "bboxes": [  
    {"box_2d": [ymin, xmin, ymax, xmax], "label": "object name"},  
    ...  
  ],  
  "predicates": [  
    {"name": "predicate_name", "args": ["object1", "object2"]},  
    {"name": "predicate_name", "args": ["object1"]},  
    ...  
  ]  
}
```

Use the object labels you detect in Task 1 when creating predicates in Task 2.

Only reference objects that you actually detected in the image.

When a human phase was supposed to have produced a new object (i.e., if the human phase makes an occluded object visible), the following block is substituted for `{extra_objects}`, naming what to look for.

A step of this task has already been carried out, and it should have left the following behind. Look for each one and, if you can see it, use EXACTLY the name given here. If one is not visible, leave it out -- do not label something else with its name.

- `{name of an object a phase should have produced}`

b) Phase segmentation: Given one workspace image, the instruction and the detected objects, it returns the ordered robot/human phases, their effect atoms, and, for every human phase, the operator it stands for including its preconditions, add effects, and delete effects.

A robot and a human share a workspace. Break the instruction below into an ORDERED list of phases. Each phase is done either by the robot or by the human, and they happen in the order you give.

THE INSTRUCTION:

`{instruction}`

The image shows the workspace as it is right now. Plan the INSTRUCTION -- the picture is context for where things are, not a task in itself.

The workspace contains exactly these objects RIGHT NOW, and no others:

`{objects}`

(If the instruction goes on to talk about something that is not there yet because the human has to produce it first, see NEW OBJECTS below. That is the one way to name something not on this list.)

The robot can do one thing: pick an object up and place it on a surface. That is all. It plans and executes each of its phases itself; you only say what must be TRUE when the phase is finished, using these predicates:

- `On(?obj: movable, ?surface: surface): {0}` is resting on top of {1}
- `Holding(?obj: movable):` the robot's gripper is holding {0}
- `HandEmpty():` the robot's gripper is empty

WHAT On CANNOT SAY. `On({0}, {1})` means {0} is RESTING LOOSELY somewhere on top of {1}, and nothing more. The robot places by opening its gripper above a surface, so it cannot fit, insert, slot, thread, plug, screw, seat, close, or align one thing to another, and On cannot ask it to. If a clause needs the object to end up IN something, or in a particular position or orientation on it -- a puzzle piece in its matching cut-out, a lid seated on a jar, a plug in a socket, a book squared onto a shelf -- that is a HUMAN phase, however much it looks like a pick-and-place. Say so with an invented predicate, not with On.

The human can do anything the robot cannot -- open, close, fold, unfold, tie, flatten, rotate, manipulate cloth. Give a human phase 'instructions' addressed to the person, and 'atoms' saying what should be true afterwards. That is what a camera will be used to check, so it must be visible.

EVERY HUMAN PHASE IS AN OPERATOR, AND YOU MUST SAY WHAT IT IS. Give the phase an 'operator' with:

- 'name': what the action is, one word, capitalised -- Push, Open, Fold, Insert, Tie, Solve.
- 'args': the objects it acts on, from the object list, in the order the name reads.
- 'preconditions': what must ALREADY be true for the person to be able to do it. Write them with On, Holding, HandEmpty or a predicate you invented. HandEmpty() belongs here whenever the person needs the robot to be out of the way -- which is almost always.
- 'add_effects': what becomes true. Every atom you put in the phase's 'atoms' must appear here.
- 'delete_effects': what STOPS being true. This is the one most often forgotten. If the person moves something off a surface, the old `On(...)` is a delete effect; if they close what an earlier phase opened, the `IsOpen(...)` is. An empty list is fine and means "this takes nothing away" -- but say it.

Worked operator. Objects: box, table. Phase: the human shoves the box aside.

```
operator: {"name": "Push", "args": ["box"],
  "preconditions": [{"predicate": "HandEmpty", "args": []},
    {"predicate": "On",
      "args": ["box", "table"]}],
  "add_effects": [{"predicate": "IsPushedAside",
    "args": ["box"]}],
  "delete_effects": [{"predicate": "On",
    "args": ["box", "table"]}]}
```

Read it as: the robot must not be holding anything and the box must be on

the table; afterwards the box is pushed aside and is no longer where it was.

The preconditions and effects are CHECKED against a camera image, before and after the person acts, and they are checked against each other across your whole plan. So they have to be true statements about the world, not decoration: do not list a precondition that nothing in your plan makes true, and do not delete something a later phase still needs.

NEW OBJECTS. Sometimes the instruction is about a thing that is not a separate object yet, and only becomes one because of what the human does: a block still inside the tower, a card still in the deck, a lid still on the jar. It is not in the list above because it cannot be seen yet.

Declare it in 'new_objects', giving its 'name', the 'created_by_phase' index of the HUMAN phase that brings it into existence, and a 'description' of what it will look like once it does. That phase must also say where it ends up -- On(<the new object>, <something from the list above>) among its atoms -- because that is how it is found in the camera image afterwards. From then on it is an ordinary object, and a ROBOT phase can pick it up and place it like anything else.

Reach for this whenever the instruction goes on to MOVE the thing the human produced. Writing that step as another human phase because you had no name for the object gives the robot's own work away.

How to divide the work:

- Give the robot every pick-and-place. A human phase that includes moving an object from A to B is taking the robot's work away from it.
- Use a human phase only for something the robot genuinely cannot do.
- ORDER MATTERS AND IS YOURS TO SET. If the box must be opened before anything can go in it, the human's "open the box" phase comes BEFORE the robot's "put the toy in the box" phase. If a cloth is folded over a toy, the robot places the toy first and the human folds afterwards. Think about what has to be true for the next phase to be physically possible.
- Intermediate states are fine and often necessary. "Take the toy off the box, open the box, put the toy back in" is three phases, and the first one ends with the toy somewhere else -- On(toy, table). Each robot phase is planned fresh, so an object may be picked up in more than one phase.
- Do not add phases the instruction does not ask for, and do not merge two of its steps into one.

PLAN THE WHOLE INSTRUCTION. Work through it clause by clause and give every clause a phase, in the order stated. Fill in 'coverage' with one entry per clause, naming the phase that carries it out (the index in your 'phases' list), or -1 if you had to leave it out. The most common mistake is to plan the first clause carefully and stop; the last phase must leave the workspace as the END of the instruction describes.

Worked example. Objects: red_toy, cardboard_box. Instruction: "take the toy off the box, open the box, then put the toy inside it". That is three clauses, so three phases:

```
phase 0, robot -- "take the toy off the box"
                  atoms: On(red_toy, table)
phase 1, human -- "open the box"
                  atoms: IsOpen(cardboard_box)
                  instructions: "Open the cardboard_box and fold its
                                flaps back."
phase 2, robot -- "put the toy inside the box"
                  atoms: On(red_toy, cardboard_box)
coverage: [{"take the toy off the box", 0}, {"open the box", 1},
           ["put the toy inside it", 2]]
```

Note the order: the box is opened BEFORE anything is put in it, the robot does both pick-and-places, and the toy is picked up in two different phases, which is allowed.

A second worked example, this time with a new object. Objects: jenga_tower, screwdriver, white_paper. Instruction: "remove a block from

the jenga tower using the screwdriver onto the white paper, and then place the block on top of the jenga tower". The loose block is not in the object list -- it is still a brick inside the tower -- so declare it:

```
new_objects: [{"name": "loose_block", "created_by_phase": 0,
               "description": "the single wooden block the human pushes
               out of the tower"}]
phase 0, human -- "push a block out of the tower onto the paper"
               atoms: On(loose_block, white_paper)
               instructions: "Use the screwdriver to push one block
               out of the jenga_tower and put it on the white_paper."
phase 1, robot -- "put the block on top of the tower"
               atoms: On(loose_block, jenga_tower)
coverage: [{"remove a block ... onto the white paper", 0},
           ["place the block on top ...", 1]]
```

Phase 1 is a ROBOT phase. It is one pick and one place, which is exactly what the robot is for; the human is needed for phase 0 only because prying a block out with a screwdriver is not a pick-and-place.

A third worked example, about that last point. Objects: pink_toy, puzzle_board, yellow_cloth. Instruction: "place the toy on the cloth and solve the puzzle". Two clauses, so two phases:

```
new_predicates: [{"name": "IsSolved", "instructions": "every piece of {0}
               is sitting down inside its own matching cut-out, flush
               with the board, with no gaps"}]
phase 0, robot -- "place the toy on the cloth"
               atoms: On(pink_toy, yellow_cloth)
phase 1, human -- "solve the puzzle"
               atoms: IsSolved(puzzle_board)
               instructions: "Fit each puzzle piece into its matching
               cut-out in the puzzle_board so it sits flush."
               operator: {"name": "Solve",
                         "args": ["puzzle_board"],
                         "preconditions": [{"predicate": "HandEmpty",
                                             "args": []}],
                         "add_effects": [{"predicate": "IsSolved",
                                             "args": ["puzzle_board"]}],
                         "delete_effects": []}]
coverage: [{"place the toy on the cloth", 0}, ["solve the puzzle", 1]]
```

"solve the puzzle" is NOT On(pink_toy, puzzle_board). Resting the toy on the board solves nothing -- the piece has to go INTO its slot, which the robot cannot do. Writing it as a robot phase is worse than useless: the robot picks the toy straight back up and drops it on the board, undoing phase 0 to achieve nothing.

Rules, all of which are checked:

- A ROBOT phase's atoms may use ONLY On, Holding and HandEmpty. The robot cannot achieve a predicate you invent -- if a phase needs one, it is a human phase.
- Give a whole pick-and-place ONE phase, ending with On(?obj, ?surface). Do not split it into a "pick it up" phase and a "put it down" phase; the robot does both as one piece of work.
- Two ROBOT phases in a row must not move the same object twice. The second placement throws the first one away, so the first is wasted motion -- and it almost always means a step that is not really a pick-and-place was given to the robot. If a human phase belongs between them, put it there; if the second phase is the one the robot cannot do, make IT the human phase.
- Every phase needs at least one atom, and a human phase needs 'instructions' and an 'operator' too.
- An operator's 'add_effects' must include every atom in its phase's 'atoms', and no atom may be in both 'add_effects' and 'delete_effects'.
- An operator's 'preconditions' must be reachable: either true in the workspace to begin with, or made true by an earlier phase. Do not require something no phase establishes, and do not delete something a later phase's preconditions still need.
- Every object name must be one of the objects listed above, spelled exactly, or one you declared in 'new_objects'. Do not name an object any other way.

- A 'new_objects' entry must be created by a HUMAN phase, that phase must come before every phase that uses the object, and that phase must place it with On(<the new object>, <a listed object>).
- Invent a new predicate only for something no existing predicate can express. A new predicate is evaluated by a vision-language model looking at a camera image of the workspace, so it needs 'instructions': a description of what must be VISIBLE in the image for it to be true. Write the text with {0}, {1}, ... standing in for the arguments, in the order they are declared.

If some clause CANNOT be expressed, put it in 'unrepresented' with the reason, and leave it out of the phases. The usual reason is that it refers to something not in the object list and nothing in your plan produces it -- "pick another toy" when only one toy was detected, and no phase makes a second one. (If a human phase of yours IS what produces it, that is a 'new_objects' entry, not an unrepresented clause.) Never invent an object to satisfy a clause, and never bind it to a different object that happens to be present. Saying you could not do it is always better than quietly doing something else: a human is watching, and can put the missing object on the table and start again.

c) *Repair*: Appended to the prompt above when a proposal fails for at most 3 attempts.

Your previous response was rejected.

Your response was:

{response}

The problem was:

{error}

Try again, fixing exactly that problem and keeping everything else that was correct.

d) *Predicate classification*: This is how an invented predicate is evaluated, and it is what settles every atom of a human phase's operator contract against the camera image. Specifically, it checks its preconditions before the person acts, and its add and delete effects afterwards.

You are the perception system of a robot. Look at the image of the robot's workspace and decide whether the following statement is true right now.

Statement: {statement}

Judge only what you can see. If the workspace does not clearly show the statement to be true, it is false. Give a one-sentence reason for your answer.

C. Details of Demonstration Collection Failures

TABLE III: Failure analysis of TANDEM demonstration collection on the five tasks. For each task, we report the number of successful demonstrations collected, the number of failed episodes, and how those failures are attributed across the stages of the system: predicate and operator invention or phase switching, TAMP planning, TAMP execution, and the human operator.

Task	Successes	Failures	Failure attribution			
			Invention / switch	TAMP (planning)	TAMP (execution)	Human
Cover Bread Rolls	20	28	0% (0/28)	3.6% (1/28)	96.4% (27/28)	0% (0/28)
Solve Constrained Puzzle	20	3	0% (0/3)	0% (0/3)	100% (3/3)	0% (0/3)
Sort & Cover Snacks	20	12	0% (0/12)	0% (0/12)	100% (12/12)	0% (0/12)
Open Obstructed Book	20	21	0% (0/21)	19.0% (4/21)	81.0% (17/21)	0% (0/21)
Store Bread in Closed Box	20	50	0% (0/50)	32.0% (16/50)	64.0% (32/50)	4.0% (2/50)
Total	100	114	0% (0/114)	18.4% (21/114)	79.8% (91/114)	1.8% (2/114)

D. Policy Execution Failure Analysis

TABLE IV: Failure analysis of policy execution on the five tasks. For each task and approach we report the number of failed evaluation trials out of 20 and how those failures are attributed across six failure modes: failing to grasp an object, failing at the subsequent manipulation or placement, drifting into a joint configuration misaligned with the scene, acting on the wrong object or stage, stalling without progress until the time limit, and running out of time before the task is finished. Percentages are taken over the failed trials of that row. For each row the most common failure mode is **highlighted** and the runner-up is underlined; ties for the runner-up are all underlined.

Task	Approach	Failures	Failure attribution					
			Grasping	Manipulation / placement	Joint-space misalignment	Semantic	Timeout (stuck)	Timeout (unfinished)
Cover Bread Rolls	$\pi_{0.5}$ -DROID	20	0%	5.0% (1/20)	0%	95.0% (19/20)	0%	0%
	HITL-TAMP	14	71.4% (10/14)	<u>28.6% (4/14)</u>	0%	0%	0%	0%
	TANDEM	11	0%	0%	9.1% (1/11)	72.7% (8/11)	<u>18.2% (2/11)</u>	0%
Solve Constrained Puzzle	$\pi_{0.5}$ -DROID	20	0%	15.0% (3/20)	0%	85.0% (17/20)	0%	0%
	HITL-TAMP	20	<u>15.0% (3/20)</u>	85.0% (17/20)	0%	0%	0%	0%
	TANDEM	10	50.0% (5/10)	<u>40.0% (4/10)</u>	0%	10.0% (1/10)	0%	0%
Sort & Cover Snacks	$\pi_{0.5}$ -DROID	20	0%	5.0% (1/20)	<u>5.0% (1/20)</u>	90.0% (18/20)	0%	0%
	HITL-TAMP	17	<u>47.1% (8/17)</u>	52.9% (9/17)	0%	0%	0%	0%
	TANDEM	5	60.0% (3/5)	<u>20.0% (1/5)</u>	0%	0%	0%	<u>20.0% (1/5)</u>
Open Obstructed Book	$\pi_{0.5}$ -DROID	20	0%	30.0% (6/20)	0%	70.0% (14/20)	0%	0%
	HITL-TAMP	14	7.1% (1/14)	85.7% (12/14)	0%	0%	0%	<u>7.1% (1/14)</u>
	TANDEM	10	40.0% (4/10)	<u>30.0% (3/10)</u>	0%	<u>30.0% (3/10)</u>	0%	0%
Store Bread in Closed Box	$\pi_{0.5}$ -DROID	20	0%	55.0% (11/20)	5.0% (1/20)	<u>40.0% (8/20)</u>	0%	0%
	HITL-TAMP	18	<u>27.8% (5/18)</u>	72.2% (13/18)	0%	0%	0%	0%
	TANDEM	3	<u>33.3% (1/3)</u>	66.7% (2/3)	0%	0%	0%	0%
All tasks	$\pi_{0.5}$ -DROID	100	0%	<u>22.0% (22/100)</u>	2.0% (2/100)	76.0% (76/100)	0%	0%
	HITL-TAMP	83	<u>32.5% (27/83)</u>	66.3% (55/83)	0%	0%	0%	1.2% (1/83)
	TANDEM	39	33.3% (13/39)	<u>25.6% (10/39)</u>	2.6% (1/39)	<u>30.8% (12/39)</u>	5.1% (2/39)	2.6% (1/39)

We complement the demonstration-collection analysis of Appendix C with an analysis of how the downstream policies fail at evaluation time. For every failed trial of the 20 evaluation trials per approach per task, we label the failure with one of six modes. Table IV reports the resulting attribution.

The three approaches fail in distinct ways. Failures of $\pi_{0.5}$ -DROID are dominated by semantic errors (76.0%): without task-specific fine-tuning the policy manipulates objects competently but pursues the wrong object or the wrong stage of the task. HITL-TAMP shows the opposite pattern, with no semantic failures but 98.8% of its failures split between grasping (32.5%) and the subsequent manipulation or placement (66.3%), reflecting that TAMP supplies the task structure while the local diffusion policy must execute the contact-rich stages. TANDEM failures are spread more evenly across grasping (33.3%), semantic errors (30.8%), and manipulation or placement (25.6%), and are far fewer in absolute terms: 39 failed trials in total, compared with 83 for HITL-TAMP and 100 for $\pi_{0.5}$ -DROID.

E. Details of the TAMP and Teleoperation Portions

Table V reports how each demonstration collected by TANDEM is divided between the segments executed by the TAMP system and the segments executed by the human teleoperator. Four of the five tasks follow a single handover, where TAMP performs the opening stages and the human completes the remainder. Store Bread in Closed Box instead hands control back to TAMP after the human stage, so a single demonstration alternates between the two modes twice. The human portion ranges from 22.3% of the trajectory on Cover Bread Rolls, where only the final covering motion lies outside the planning domain, to 65.5% on Open Obstructed Book, where the contact-rich book opening dominates the episode.

TABLE V: Composition of the demonstrations collected by TANDEM. For each task we report the number of demonstrations n , the order in which control alternates between TAMP and the human teleoperator, the share of the trajectory duration spent in each portion, and the mean duration of a complete trajectory. A dash indicates that the task has no second TAMP portion.

Task	n	Structure	Share of trajectory duration			Mean traj.
			TAMP (1)	Teleop	TAMP (2)	
Cover Bread Rolls	80	TAMP $\rightarrow$ Teleop	77.7%	22.3%	—	58.4 s
Solve Constrained Puzzle	20	TAMP $\rightarrow$ Teleop	44.2%	55.8%	—	32.7 s
Sort & Cover Snacks	20	TAMP $\rightarrow$ Teleop	55.2%	44.8%	—	47.2 s
Open Obstructed Book	20	TAMP $\rightarrow$ Teleop	34.5%	65.5%	—	40.4 s
Store Bread in Closed Box	20	TAMP $\rightarrow$ Teleop $\rightarrow$ TAMP	24.5%	37.0%	38.5%	60.7 s